CLAUDE CODE / TOOL DESIGN / CHINESE VERSION

像 Agent 一样看世界

Claude Code 团队如何从模型视角设计工具：什么时候该加工具，什么时候该删掉它。

ANTHROPIC BLOG · 2026.04.10 · THARIQ SHIHIPAR

文章先抛出核心问题：Agent 到底需要什么样的工具

问题不是越多越好

- Claude 完全通过 **tool calling** 行动
- 你可以只给一个通用工具，也可以给几十个专用工具
- 真正难的是：**工具形状要贴合模型能力**

作者的建议

观察输出、频繁实验、不断修正你对模型能力边界的判断。

关键词

see like an agent

先理解模型怎样思考，再决定给它什么工具。

工具设计不是 API 列表问题，而是一次关于“模型如何完成任务”的认知建模。

作者用一道数学题来解释：工具必须顺着模型的能力曲线

纸和笔
最低可用，但复杂计算会被人工步骤拖慢。

计算器
更快，但前提是你知道高级功能该怎么操作。

计算机
最强，但前提又变成你会写代码、会执行代码。

> 同一个任务，最适合的工具集合，取决于使用者自身的技能结构。

案例一：AskUserQuestion 不是一次到位，而是三次试错

ATTEMPT 1

把问题塞进 ExitPlanTool

计划和提问混在一起，让 Claude 困惑：答案若冲突，是否要再调一次工具？

ATTEMPT 2

改成 Markdown 格式约束

模型能大致输出，但不稳定，常常多说一句、漏选项、格式走样。

ATTEMPT 3

独立的 AskUserQuestion Tool

让模型在任意时点结构化提问，UI 可承载选项，用户回答也能阻塞 agent 循环。

文章强调的判断标准很实用：**再好的工具，如果模型不理解怎么调用，就等于没设计好。**

案例二：模型变强后，旧工具反而可能变成束缚

早期做法

- 用 **TodoWrite** 维持任务清单
- 每 5 轮系统再提醒一次目标
- 问题是 Claude 容易把 todo 当成不能修改的死列表

后来的替代

随着模型更擅长改计划、派 subagent、协调依赖，团队把重点从“别跑偏”换成“让 agent 之间能沟通”。**于是 TodoWrite 被 Task tool 取代。** 它支持依赖、共享更新、修改与删除，更适合多 agent 协作。

一个重要启发是：**你昨天为了补能力缺口加的工具，明天可能会限制新能力发挥。**

最关键的工具，是让 Agent 自己找到上下文

阶段 1

RAG 预取上下文
快，但依赖索引，且上下文是系统塞给 Claude 的。

阶段 2

Grep 搜代码库
不再只在网页上搜索，而是让 Claude 自己在 repo 里建立上下文。

阶段 3

Skills + 渐进披露
skill 文件还能继续引用别的文件，形成多层嵌套探索。

结论

随着模型变强，最好的上下文往往不是“提前喂给它”，而是“让它自己找出来”。

渐进披露的价值：增加能力，不一定要增加工具

方案 A

塞进系统提示

所有知识都常驻上下文，会造成 context rot，也会干扰主任务。

方案 B

直接搜文档

可行，但主 agent 会把大量文档拉进上下文，只为回答一个小问题。

方案 C

Claude Code Guide 子代理

让专门 subagent 搜文档、提炼答案，再回传给主 agent，保持主上下文干净。

这页的重点不是“又造了一个工具”，而是通过子代理和技能，把能力扩张放进可控的上下文边界里。

最后收束成四条设计原则

贴合能力

工具形状应匹配模型当前会不会用

持续淘汰

模型变强后，旧工具可能该删而不是继续补

自建上下文

尽量让 AGENT 搜索、发现、递归读取

控制边界

用渐进披露和子代理避免主上下文膨胀

这篇文章真正想说的是：**tooling 设计不是静态配置，而是要随着模型能力一起演化。**